1. Контекст проекта

1.1 Бизнес-задача

Описание проблемы/вызова, который пытаемся решить:

Обычным гражданам, не обладающим профессиональным юридическим образованием сложно быстро находить релевантные статьи кодексов, понимать юридических язык или ориентироваться в структуре НК ТК УК и КоАП РФ. Обычный поиск в интернете часто выдает статьи без контекста, требует знания номера статьи и не дают объяснения нормы простым языком.

Почему именно ваше решение проекта имеет смысл:

LLM + RAG позволяет искать статьи по смыслу, объяснять юридический текст простым языком и ссылаться на конкретную статью/пункт, что помогает снижать барьер для понимания законодательства.

Каковы критерии успеха с точки зрения бизнеса:

- Recall@5 ≥ 85% (релевантная статья в топ-5 retrieval)
- Доля ответов с корректным цитированием ≥ 95%
- Среднее время ответа < 10 сек
- Доля положительных реакций пользователей более 80%

1.2 Целевая аудитория и пользователи

Кто будет пользоваться ботом: внутренние сотрудники, клиенты, партнёры

- Студенты и преподаватели юридических вузов для быстрого поиска релевантной информации
- Малый бизнес в случае с НК и ТК РФ
- Граждане, ищущие справочную информацию

Какие сценарии использования: текстовый чат, голосовой интерфейс, мессенджеры, веб-виджет.

Текстовый чат-бот в телеграме

Какова предполагаемая нагрузка: число сессий/пользователей, пиковые часы.

Для реального MVP-проекта:

500-1000 активных пользователей. До 500 сессий в день. Пиковые часы ожидаем в будни в послерабочее время: 18:00-21:00.
Цена ошибки довольно высокая, поэтому бот не дает индивидуальных консультаций, всегда цитирует источник, добавляет дисклеймер.

1.3 Ограничения и допущения

Что нельзя/не будет делать (например: не интегрируемся с X, не обрабатываем Y):

- Не формируем юридические документы, не прогнозируем исход судебного дела и не интерпретируем конкретные судебные кейсы - то есть не даем прямых юридических консультаций,
- Не интегрируемся с системами "Консультант+" или "Гарант" на уровне API.
- Отвечаем строго на вопросы по ТК РФ, НК РФ, КоАП РФ и УК РФ. Запросы по другим отраслям права игнорируем с предложением обратиться к другим источникам.

Предположения о данных, инфраструктуре, требованиях к безопасности, к нормативам:

- Источники официальные и открытые - используем тексты кодексов из официальных/публичных источников (без платных баз, без закрытых комментариев). Каждый документ имеет ссылку на источник и дату выгрузки.
- Для MVP используем “актуальную редакцию на дату выгрузки”. В качестве расширения предполагаем, что еженедельно обновлять актуальность.
- Кодексы имеют иерархию “раздел → глава → статья → часть/пункт”. Предполагаем что эта структура стабильна, ее можно восстановить и сохранить как метаданные
- Тексты доступны в HTML/структурированном виде или PDF хорошего качества.
- Нет чувствительных данных - база знаний не содержит персональных данных. Пользовательские запросы могут содержать личные детали — поэтому логирование будет анонимизированным (см. ниже).
- Язык - русский

Предположения об инфраструктуре

- Развёртывание возможно локально/на одном сервере. MVP работает на одном CPU). Контейнеризация Docker обязательна для воспроизводимости.
- Есть доступ к LLM через API (или к локальной модели), с ограничениями по стоимости/лимитам.В MVP допускаем облачную LLM, потому что в базе знаний нет секретных данных.
- Используем FAISS локально – предполагаем, что объём чанков укладывается в память/диск одного сервера.
- Тексты и метаданные можно хранить в:

- файловой системе (JSON/Parquet) для MVP или Postgres
- Индекс embeddings хранится отдельно и пересобирается при обновлении корпуса.
- Логирование и мониторинг:
- структурированные логи (JSON)
- сохранение ошибок/латентности
- предполагаем, что есть место для хранения логов
- **Безопасность** - данные пользователя не используются для обучения модели.

2. Архитектура решения

2.1 Общая схема

Будем опираться на стандартную RAG-архитектуру:

1. Пользователь взаимодействует с фронт-эндом (в нашем случае чат в телеграм месенджере), который отправляет сообщения на API.
2. API - это тонкий слой (например, FastAPI), принимающий запросы, проверяющий базовые ограничения (rate limiting, валидация формата) и передающий управление в core - основной сервис оркестрации.
3. Core - основа системы, здесь принимаются решения о том, как обработать вопрос, какие данные достать, как собрать промпт и как оформить ответ.
4. База/хранилище знаний хранит исходные тексты документов и их метаданные (например, кодекс, статья, часть/пункт, дата редакции, источник). Для MVP это может быть файловое хранилище (JSONL/Parquet) или Postgres. Отдельно от этого хранится векторный индекс (FAISS или pgvector): в нём лежат embeddings для каждого чанка текста и указатели на соответствующие chunk_id. То есть обычная база хранит читаемый текст + метаданные, а векторная - вектора для поиска по смыслу.
5. Мы ходим в хранилище дважды: на этапе retrieval - делаем поиск top-k релевантных чанков по embedding запроса, далее после выдачи ответа - сохраняем технические логи (latency, какие чанки использованы, флаги отказа/ошибки), при этом без хранения чувствительных данных.
6. Промпт не хранится в базе знаний: он хранится как часть кода/конфигурации сервиса core (в репозитории, можно с версионированием). Он представляет собой набор шаблонов: системные инструкции (политика безопасности и стиль), формат подачи контекста (как вставлять найденные фрагменты), и шаблон ответа (обязательное цитирование статей). Это важно, потому что промпт меняется как “логика продукта” и должен быть контролируемым через git и релизы.

Порядок обработки пользовательского запроса следующий. Сначала фронт-энд отправляет текст сообщения в API. API проверяет лимиты, формат и прокидывает запрос в core. Core выполняет предобработку: нормализует текст, определяет язык. Далее core запускает retrieval: преобразует вопрос в embedding и делает запрос в

векторный индекс, получая top-k релевантных чанков, при этом может применяться фильтрация по метаданным (например, выбирать только нужный кодекс или редакцию). Если релевантных фрагментов нет или они слишком слабые, core возвращает отказ. Если контекст найден, core собирает промпт: включает инструкции, добавляет найденные фрагменты документов (с указанием статьи/пункта и источника), и вставляет вопрос пользователя. Затем core вызывает LLM (через API) и получает черновик ответа. После этого срабатывает пост-обработка: проверяется, что ответ содержит ссылки на статьи/пункты, не содержит запрещённых формулировок (например, “гарантированно выиграете суд”), при необходимости добавляется дисклеймер и блок “Источники”. На этом этапе также могут применяться простые защитные эвристики: если ответ не опирается на контекст или не содержит ссылок, он заменяется на безопасный отказ/уточнение. Финальный ответ возвращается через API во фронт-энд, а технические метрики и информация о retrieval логируются для последующего анализа качества и стабильности системы.

2.2 Хранилище знаний / документооборот

Источники:

- Трудовой кодекс РФ
- Налоговый кодекс РФ (части 1 и 2)
- Кодекс об административных правонарушениях
- Уголовный кодекс РФ (УК РФ)

Чистый текст будет извлекаться из PDF с сохранением структуры (Номер статьи, название, часть, пункт). Нормализация: приведение к единому формату, удаление лишних пробелов, спецсимволов. Категоризация путем добавления метаданных: `{ "codex": "TK", "chapter": 11, "article": 123, "title": "Ежегодный отпуск" }`.

Текст каждого блока прогоняется через модель эмбеддингов.

Данные могут обновляться раз в неделю или при появлении поправок.

2.3 Интеграции и интерфейсы

- Основная схема:
  Telegram Bot API для работы с Telegram-клиентом: клиент отправляет запрос, после чего находятся наиболее актуальные документы из БД и формируется запрос LLM-модели. После этого формируется ответ, который должен получить пользователь, если модель достаточно уверена в ответе, то ответ отправляется в чате
- UI/UX:
- Простой интерфейс: поле ввода и история сообщений, обычный ТГ-чат
- Под каждым ответом блока кнопки 👍 / 👎 для обратной связи (собираем инфу о клиентском опыте)
- Логирование:

Каждый запрос/ответ логируется в БД. Фиксируются время, ID пользователя, текст запроса, текст ответа, ID найденных документов, оценка пользователя.
- Метрики: В будущем (не MVP), когда появятся первые пользователи, планируется мониторинг MAU / DAU / количество запросов по дням. Если получится, реализуем логирование через LangSmith
- Алерты:
  Если растет время ответа или количество обращений в тех. поддержку, то в будущем планируется добавление алертов (не MVP версия)

2.4 Инфраструктура и развертывание

Проект разворачивается как набор Docker-контейнеров для воспроизводимости и простого запуска на одном компьютере.

- Backend: Python + FastAPI (REST API для чата)
- RAG/ретривал: FAISS (локальный векторный индекс)
- LLM: вызов модели Yandex GPT через API
- Хранилище документов: файлы на диске (JSONL/Parquet) + метаданные (в тех же файлах или SQLite/Postgres)
- UI: Тг-бот

Kubernetes/оркестрация/серверлесс в MVP не используются, так как проект демонстрационный и разворачивается на одном узле.

Требования к вычислениям

Проект рассчитан на запуск на CPU.

- CPU: достаточно 2-4 vCPU
- RAM: 4-8 GB (FAISS + приложение + кэш)
- Хранилище: 1-2 GB для корпуса и индекса (зависит от чанков и embedding-модели)

GPU не требуется, так как генерация выполняется внешней LLM через API, а embeddings строятся офлайн.

Безопасность (минимально необходимое)

Так как документы публичные (кодексы), требования проще, но базовые меры всё равно есть:

- Rate limiting (например, 10 запросов/мин на IP) — защита от спама и перерасхода API
- Секреты (ключи LLM) хранятся в .env, не коммитятся в репозиторий
- Логи: не храним персональные данные; если запрос может их содержать - маскируем email/телефон в логах

GDPR/комплаенс детально не рассматриваем, потому что система не обрабатывает персональные данные и не хранит пользовательские профили

3. Данные и качество знаний

В данном проекте используются только тексты законов, которые находятся в общем доступе и не являются чувствительными / конфиденциальными данными. В боте пропишем предупреждение, чтобы пользователи не отправляли свои конфиденциальные данные.

3.1 Сбор и предобработка данных

- Источники документов: законодательные документы (кодексы), опубликованные на сайте КонсультантПлюс (скорее всего, будем)
- Шаги предобработки:

1) Извлекаем текст из pdf-документов с помощью библиотеки pdfplumber
2) Очищаем от рекламы, колонтитулов, номеров страниц
3) Разбиваем текст на чанки, где один чанк = одна статья
4) Выделяем именованные сущности (МВД, НДС и тд), ключевые слова, метаданные (название кодекса + номер статьи + название статьи) - например, с помощью библиотек natasha, spacy
5) Для токенизации используем токенизатор выбранной LLM

3.2 Векторизация и индексирование

- Для создания эмбеддингов можно использовать BAAI/bge-m3 (длинное контекстное окно, есть поиск по смыслу и по ключевым словами)
- Параметры индекса: размер - 1024
- Для векторного поиска попробуем FIASS, если не получится, то в качестве метрики возьмем cosine similarity, в качестве алгоритма будем использовать hierarchical navigable small world, тк это один из самых быстрых и точных алгоритмов для векторного поиска
- Шарды и реплики не потребуются
- Стратегии обновления: поскольку у нас относительно небольшой корпус документов, проще использовать полную пересборку

3.3 Метрики качества знаний

- Покрытие: мы создаем навигатор по кодексам и используем в качестве источников все тексты кодексов, поэтому технически у нас 100%-ное покрытие. Что касается сценарного покрытия, мы можем рассмотреть самые популярные юридические запросы пользователей (например, в Яндексе или в Google), загрузить их в нашего бота и посчитать долю кейсов, когда он предложил верную статью.
- Актуальность: база будет обновляться 1 раз в неделю, в каждом сообщении бота мы будем указывать дату, на которую актуальна предоставляемая информация
- Консистентность: благодаря полной пересборке мы решаем многие проблемы с дубликатами и противоречиями. Чтобы обработать ситуацию, в которой есть несколько законов, подходящих под кейс пользователя, в промте пропишем, что нужно перечислить все подходящие статьи законов
- Проверка:

1) Для ручной ревизии мы создадим набор наиболее типичных кейсов и мер пресечения/решений, соответствующих актуальному законодательству (или можем выгрузить несколько судебных дел и вынесенных по ним решений) и будем периодически прогонять бота по этим кейсам, чтобы оценить его на здравый смысл
2) Для спотыкаемых примеров мы аналогично наберем пул граничных кейсов, когда неясно, какую статью закона стоит применить к ситуации. Наш бот должен будет выдать все релевантные статьи.

4. Модель и генерация

4.1 Выбор LLM и промптинг

- Модель:Yandex GPT, чтобы было более точное цитирование + ссылки на источники и опора на них (а не придумывание информации) и возможность использовать API
- Промпт-шаблон: Ты – эксперт-юрист, навигатор по законодательству РФ. Ответь на запрос пользователя, используя информацию только из источников ниже. Нельзя давать индивидуальные юридические рекомендации. Если в контексте нет ответа на вопрос, не хватает информации, ты не уверен или запрос не относится к источникам, ответь: “К сожалению, у меня нет информации на этот вопрос. Попробуйте переформулировать”. В конце обязательно укажи ссылки на используемые источники из контекста с цитатами. Запрос пользователя: “...”.
- Максимальная длина: 2000 токенов, чтобы не множить галлюцинации и не плодить пересказ кодексов, но при этом хватает на цитирование и ссылки на источники
- Скорость генерации: Около 10 секунд на ответ (предварительно отправка сообщения “Ищу ответ на Ваш запрос…”, чтобы пользователь не подумал, что все зависло)

4.2 Контроль качества ответов

Метрики:

1) Точность – % ответов с источниками, которые соответствуют фактам
2) Полнота – % ответов, когда бот реально дал ответ, когда его нужно было дать
3) Полезность / удовлетворенность по мнению пользователей – соотношение лайков-дизлайков, которые ставят пользователи

Оценка ошибок:

1) Галлюцинации – возможны, но если будем везде писать, что это не является индивидуальной юридической документацией + делать ссылки на конкретные статьи, на которые опиралась система при ответе, то не должно вызвать сильного пользовательского негатива (система – это навигатор по кодексам, а не замена налогового консультанта)
2) Офтопик – модель должна уметь “не отвечать” на вопросы, которые не относятся к ее сфере, и уходить в заглушку. В том числе модель должна уходить в заглушку, если она не уверена в ответе

Проверять и тестировать систему будем ручной проверкой.

4.3 Обучение/дообучение

Не предусмотрено

5. UX / пользовательский опыт

5.1 Сценарии взаимодействия

- Основной сценарий:

1) Пользователь задает вопрос
2) Бот выдает сообщение о начале обработки запроса
3) Бот ищет информацию
4) Бот выдает ответ одним сообщением с ссылкой на кодекс и статью
5) Выдает кнопки “лайк/дизлайк” (либо шкалу от 1 до 5), чтобы пользователь оценил качество

- Исключительный сценарий:

Ситуации, когда нет ответа или не поняли, можем определить например по скору уверенности модели. В таком случае выдадим сообщение: “Ваш вопрос не относится к профилю данного навигатора, проконсультируйтесь с релевантными источниками”.

5.2 Диалоговая логика

- В качестве MVP используем single-turn режим, не храним историю и не ведем контекст
- Отвечаем в нейтральном тоне, используем официально-деловой стиль, структурированные и понятные ответы

- Итоговая LLM не должна использовать эмодзи в тексте; для микроинтеракций используем сообщения вида “Бот печатает сообщение…”, чтобы во время обработки запроса пользователь не думал, что бот завис
- Также добавим возможность для пользователя оценить сообщения с помощью эмодзи “лайк/дизлайк”

5.3 Метрики UX

- Будем измерять время до первого ответа, оно должно быть <10 секунд
- Также будем измерять рейтинг полезности ответа с помощью бинарных стикеров “лайк/дизлайк”
- Еще можно измерять retention - задают ли люди боту вопросы после получения первого ответа
- Для логирования берем следующие сущности: ID пользователя, текст его вопроса, список ID статей, которые нашел векторный поиск, финальный ответ LLM, затраченное время

6. Безопасность, соответствие и этика

- Обработка данных: Не собираем ФИО, email, телефон, только ID пользователя в Telegram
- Плохой контент: Фильтр на входе (не отвечаем на вопросы, не относящиеся к кодексам) и на выходе (если модель не уверена / сомневается, выдаем заглушку)
- Прозрачность: В каждом ответе есть ссылка на первоисточник + есть дисклеймер: "Этот бот является инструментом для поиска информации и навигатором по кодексам, а не заменой квалифицированной юридической помощи"
- Регулятивы: Не будем использовать персональные данные пользователя.

7. План внедрения и эксплуатации

7.1 Этапы проекта

- Фаза 0: Исследование и документирование: выбор стека технологий, сбор данных
- Фаза 1: MVP: Скачивание и индексация одного кодекса (например, ТК РФ). Разработка простого Telegram-бота. Ручное тестирование
- Фаза 2: Добавление остальных кодексов (НК, КоАП, УК). Реализация политики обработки некорректных сообщений / некорректных ответов модели. Ручное тестирование

7.2 Поддержка и эксплуатация

После реализации MVP и появления первых пользователей предусмотрен мониторинг основных метрик (MAU / DAU, количество запросов и т. д.). Обновление документов происходит раз в неделю, чтобы минимизировать риск ошибки модели из-за неверной информации в источнике. Можно также указать контакт технической поддержки и сделать алерты на превышение количества запросов технической поддержке и времени ответа системы пользователю (после реализации MVP)

8. Риски и допущения

1) Ошибки модели, которые выдают неправильный ответ на ситуацию пользователя: например, выдают ссылки на нерелевантные статьи или отвечают на запрос вне компетенций системы. Вероятность средняя (с учетом редакции промпта и указания использовать только определенные источники), влияние высокое, стратегия уменьшения – дообучение
2) Устаревшие редакции законов (в MVP фиксированный список, обновляется раз в неделю). Вероятность средняя, влияние высокое, стратегия уменьшения – более частое обновление в следующих версиях
3) Низкое попадание в нужные статьи. Вероятность средняя, влияние среднее, стратегия уменьшения – более качественный chunking, увеличение числа источников, которые отправляются с доработанным промптом

Основное допущение – используемую LLM не надо будет дообучать

9. Бюджет и ресурсы

Человеческие ресурсы: 4 студентки бэад + 40-80 часов упущенного сна на весь коллектив
ROI оценки 100% (ожидаем 10/10)

10. Приложения

Словарь терминов и аббревиатур

LLM (Large Language Model) — большая языковая модель, используемая для генерации текста.

RAG (Retrieval-Augmented Generation) — архитектурный подход, при котором модель генерирует ответ на основе найденных релевантных документов.

Embedding — числовое представление текста в виде вектора, позволяющее выполнять поиск по смыслу.

Vector DB (векторная база данных) — система хранения embeddings для быстрого поиска по косинусной близости.

Chunk — фрагмент документа (в проекте — статья или часть статьи кодекса), используемый как единица индексации.

Retrieval — этап поиска релевантных фрагментов в базе знаний.

Hallucination (галлюцинация) — генерация моделью информации, отсутствующей в источниках.

Recall@k — метрика качества поиска, отражающая, найден ли релевантный документ среди k первых результатов.

MRR (Mean Reciprocal Rank) — метрика, учитывающая позицию релевантного документа в выдаче.

Refusal — корректный отказ системы при отсутствии достаточных данных.

Latency — время ответа системы.

Top-k — количество документов, передаваемых в модель после поиска.

Чек-лист готовности к запуску

Перед демонстрацией или релизом MVP проверяется, что retrieval возвращает релевантные статьи, ответ содержит номер статьи, ответ содержит ссылку или цитату, корректно работает отказ при отсутствии информации, дисклеймер отображается.

Планируем проверить вручную ≥ 50 тестовых вопросов, получить на этих запросах Recall@5 ≥ целевого значения, ожидаем, что не будет критических галлюцинаций. Ожидаем, что нет ошибок API.

Нужен будет чек, что логи не содержат персональных данных + корректно обрабатываются некорректные запросы.